

Understanding Hypergraph-Product qLDPC Codes and Decoding

A technical learning and implementation guide

This guide builds one connected mental model from binary parity checks to CSS stabilizers, hypergraph-product construction, syndrome extraction, and qLDPC decoding. It also maps each concept to the planned Configure, Inject, Observe, and Decode workflow of the project.

Document version	1.0
Date	September 2026
Scope	qLDPC and hypergraph-product codes only
Starting noise model	Code-capacity Pauli noise

How to use this guide. Chapters 1–5 establish the mathematics. Chapters 6–9 explain errors, syndromes, and decoding in operational terms. Chapters 10–12 turn those ideas into the software and experimental plan for the explorer. The appendices provide notation, explicit matrices for the project fixture, and implementation checklists.

Contents

1	Learning goals and project scope	4
1.1	The system boundary	4
2	From classical parity checks to LDPC structure	4
2.1	Binary linear codes	4
2.2	What low density means	5
2.3	The Tanner graph	5
3	Quantum stabilizer and CSS foundations	5
3.1	Pauli errors as binary data	5
3.2	Stabilizer codes	6
3.3	CSS codes	6
3.4	Logical operators and distance	7
3.5	What makes a code qLDPC?	7
4	The hypergraph-product construction	7
4.1	Inputs and qubit sectors	7
4.2	Check matrices	7
4.3	Why the checks commute	8
4.4	Dimension	8
4.5	Distance intuition	8
5	Worked example: the project fixture $[[13, 1, 3]]$	8
5.1	Classical input	8
5.2	Derived parameters	9
5.3	Why this is a useful teaching code	9
6	Noise models and error injection	9

6.1	Code-capacity noise	9
6.2	Manual injection	10
6.3	Seeded stochastic injection	10
6.4	Later noise-model layers	10
7	Syndromes: what the system observes	11
7.1	Calculating the two syndrome channels	11
7.2	What a syndrome does and does not reveal	11
7.3	Observe-stage representations	11
8	The quantum decoding problem	11
8.1	Decoder input and output	12
8.2	The real success condition	12
8.3	Maximum-likelihood objectives	12
8.4	Two independent binary decoders or one correlated decoder?	12
9	Decoder families relevant to HGP qLDPC codes	13
9.1	Exact lookup or exhaustive decoding	13
9.2	Belief propagation	13
9.3	BP with ordered-statistics decoding	13
9.4	Small-set-flip	14
9.5	Other useful directions	14
10	A detailed BP+OSD decoding trace	14
10.1	Inputs	14
10.2	BP phase	15
10.3	OSD phase	15
10.4	Outcome classification	15
11	Project architecture and data contracts	16
11.1	Scientific state model	16
11.2	Stage contracts	16
11.3	API responsibilities	16
12	Human-centered scientific interaction	17
12.1	Interaction principles	17

12.2 Visual semantics	17
12.3 Details on demand	17
13 Experimental methodology	18
13.1 Single-run diagnostics	18
13.2 Monte Carlo experiments	18
13.3 Threshold studies	18
13.4 Metrics beyond logical error rate	18
14 Verification and testing strategy	19
14.1 Code-construction invariants	19
14.2 Stage-level properties	19
14.3 Independent verification	19
15 Implementation roadmap	20
15.1 Current foundation: Configure	20
15.2 Next slice: Inject	20
15.3 Then: Observe	20
15.4 Then: Decode	20
15.5 Research extensions	21
16 Study questions and practical exercises	21
16.1 Concept checks	21
16.2 Matrix exercises	21
16.3 Implementation exercises	21
A Notation and glossary	22
B Explicit matrices for the $[[13, 1, 3]]$ fixture	23
C Decoder comparison worksheet	24
D Pre-experiment checklist	24

1 Learning goals and project scope

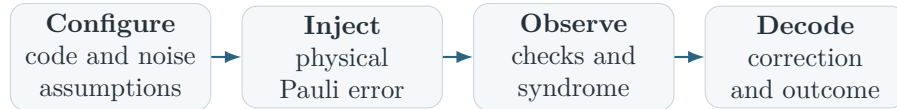
By the end of this guide, a reader should be able to:

- explain why sparse parity checks are attractive for quantum error correction;
- translate between classical parity-check matrices, Tanner graphs, and CSS stabilizers;
- construct the two HGP check matrices and verify that they commute;
- interpret a Pauli error as two binary vectors and calculate its two syndromes;
- state the correct decoding success condition in the presence of degeneracy;
- compare exact lookup, belief propagation, BP+OSD, and small-set-flip decoding;
- design reproducible qLDPC experiments with honest assumptions and useful metrics; and
- understand how these concepts become interactions in the qLDPC CoDesign Explorer.

1.1 The system boundary

The explorer is intentionally centered on qLDPC codes. It is not a surface-code comparison tool. The initial teaching fixture is a small HGP code with parameters $[[13, 1, 3]]$, constructed from two length-three repetition-code checks. Its small size allows exact verification, explicit matrices, and complete decoding traces. Later code families can be added behind the same contracts after the core interactions are trustworthy.

The planned product journey is shown below.



Each stage should expose a state transition that can be inspected and reproduced. The interface should never imply that a decoder succeeded merely because it returned a vector.

2 From classical parity checks to LDPC structure

2.1 Binary linear codes

A binary linear code is a vector subspace $C \subseteq \mathbb{F}_2^n$. A parity-check matrix $H \in \mathbb{F}_2^{m \times n}$ defines the code as

$$C = \ker H = \{c \in \mathbb{F}_2^n : Hc^T = 0\}. \quad (1)$$

If $r = \text{rank}(H)$, then the number of encoded classical bits is $k = n - r$. The code distance is the minimum Hamming weight of a nonzero codeword,

$$d = \min_{c \in C \setminus \{0\}} \text{wt}(c). \quad (2)$$

If a codeword is corrupted by an error $e \in \mathbb{F}_2^n$, the receiver observes the syndrome

$$s = He^\top. \quad (3)$$

The syndrome says which parity constraints are violated. It usually does not uniquely identify the error.

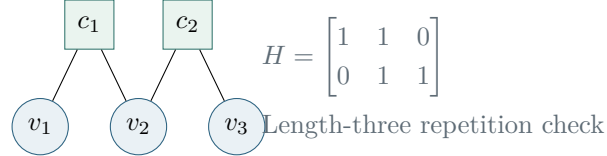
2.2 What low density means

An LDPC family has sparse parity-check matrices: check weights and qubit or bit degrees remain bounded, or at least grow very slowly, as the block length increases. Sparsity matters because it supports:

- local constraint evaluation rather than dense global checks;
- message-passing decoders with work proportional to the number of graph edges;
- bounded participation of each variable in checks; and
- concise Tanner-graph representations.

2.3 The Tanner graph

The Tanner graph of H is bipartite. Variable node v_j represents column j , check node c_i represents row i , and an edge (c_i, v_j) exists exactly when $H_{ij} = 1$.



For this matrix, the kernel contains 000 and 111, so the classical code has parameters $[3, 1, 3]$. This small check matrix will be used twice in the project fixture.

Key idea

The matrix view is ideal for algebra and validation. The Tanner-graph view is ideal for sparse computation and human explanation. The explorer should let a user move between both without changing the underlying experiment state.

3 Quantum stabilizer and CSS foundations

3.1 Pauli errors as binary data

Ignoring global phase, an n -qubit Pauli operator can be represented by two binary vectors:

$$E \longleftrightarrow (e_X \mid e_Z) \in \mathbb{F}_2^{2n}. \quad (4)$$

At qubit j , the pair $(e_{X,j}, e_{Z,j})$ means

$e_{X,j}$	$e_{Z,j}$	Pauli
0	0	I
1	0	X
0	1	Z
1	1	Y up to phase

Two Paulis $(x \mid z)$ and $(x' \mid z')$ commute exactly when their symplectic product vanishes:

$$xz'^T + zx'^T = 0 \pmod{2}. \quad (5)$$

3.2 Stabilizer codes

A stabilizer code is the joint $+1$ eigenspace of an abelian subgroup $\mathcal{S}$ of the Pauli group that excludes $-I$. If there are r independent stabilizer generators on n qubits, the codespace encodes $k = n - r$ logical qubits.

An error has a nonzero syndrome when it anticommutes with one or more measured stabilizer generators. Stabilizers themselves act trivially on the logical state. This creates the central feature of quantum decoding: many physically different errors are logically equivalent.

3.3 CSS codes

A CSS code separates stabilizer generators into X -type and Z -type checks. Let

$$H_X \in \mathbb{F}_2^{m_X \times n}, \quad H_Z \in \mathbb{F}_2^{m_Z \times n}. \quad (6)$$

Rows of H_X locate X operators in X -type stabilizers; rows of H_Z locate Z operators in Z -type stabilizers. Commutation is exactly the matrix condition

$$H_X H_Z^T = 0. \quad (7)$$

The number of logical qubits is

$$k = n - \text{rank}(H_X) - \text{rank}(H_Z). \quad (8)$$

For an error $(e_X \mid e_Z)$, the check-type convention gives

$$s_X = H_X e_Z^T, \quad s_Z = H_Z e_X^T. \quad (9)$$

X checks detect the Z component, while Z checks detect the X component. A Y error has both components and can activate both syndromes.

Syndrome naming convention

Some software names s_X after the error component being decoded, not the check type being measured. Both conventions exist. Every API, plot, and decoder trace must state its convention. The equations in this guide name a syndrome after its check type.

3.4 Logical operators and distance

An X -type operator commutes with all Z checks when its support vector lies in $\ker H_Z$. It is a trivial stabilizer when it lies in $\text{rowspan}(H_X)$. Therefore

$$d_X = \min\{\text{wt}(x) : x \in \ker H_Z \setminus \text{rowspan}(H_X)\}, \quad (10)$$

$$d_Z = \min\{\text{wt}(z) : z \in \ker H_X \setminus \text{rowspan}(H_Z)\}, \quad (11)$$

$$d = \min(d_X, d_Z). \quad (12)$$

A code with distance d corrects every adversarial error of weight at most $\lfloor (d-1)/2 \rfloor$. Under a stochastic noise model, larger errors may also be corrected; the distance guarantee is a worst-case statement.

3.5 What makes a code qLDPC?

A CSS family is qLDPC when H_X and H_Z remain sparse as n grows: stabilizer weights and the number of stabilizers touching each qubit remain bounded. The notation $[[n, k, d]]$ records block length, logical dimension, and distance, but it does not record check weight, graph degree, connectivity, decoder complexity, or measurement-circuit cost. Those must be reported separately.

4 The hypergraph-product construction

4.1 Inputs and qubit sectors

Take two classical binary parity-check matrices

$$H_1 \in \mathbb{F}_2^{r_1 \times n_1}, \quad H_2 \in \mathbb{F}_2^{r_2 \times n_2}. \quad (13)$$

The HGP construction creates two sectors of physical qubits:

$$Q_{VV} = V_1 \times V_2, \quad Q_{CC} = C_1 \times C_2. \quad (14)$$

Their sizes are $n_1 n_2$ and $r_1 r_2$, so the total block length is

$$n = n_1 n_2 + r_1 r_2. \quad (15)$$

One shared column space for H_X and H_Z

$V_1 \times V_2$ qubits
 $n_1 n_2$ columns

$C_1 \times C_2$ qubits
 $r_1 r_2$ columns

4.2 Check matrices

With a consistent Kronecker-product ordering, define

$$H_X = \left[H_1 \otimes I_{n_2} \mid I_{r_1} \otimes H_2^\top \right], \quad (16)$$

$$H_Z = \left[I_{n_1} \otimes H_2 \mid H_1^\top \otimes I_{r_2} \right]. \quad (17)$$

The dimensions are

$$H_X : (r_1 n_2) \times (n_1 n_2 + r_1 r_2), \quad (18)$$

$$H_Z : (n_1 r_2) \times (n_1 n_2 + r_1 r_2). \quad (19)$$

4.3 Why the checks commute

Multiplying the two block matrices gives

$$H_X H_Z^T = (H_1 \otimes I_{n_2})(I_{n_1} \otimes H_2^T) + (I_{r_1} \otimes H_2^T)(H_1 \otimes I_{r_2}) \quad (20)$$

$$= H_1 \otimes H_2^T + H_1 \otimes H_2^T \quad (21)$$

$$= 0 \pmod{2}. \quad (22)$$

The two equal terms cancel because arithmetic is binary. This is the algebraic heart of the HGP construction: commutation is built in rather than checked by trial and error.

4.4 Dimension

Define

$$k_i = n_i - \text{rank}(H_i), \quad k_i^T = r_i - \text{rank}(H_i). \quad (23)$$

Then the HGP code dimension is

$$k = k_1 k_2 + k_1^T k_2^T. \quad (24)$$

The transpose-code terms matter when the input checks have dependent rows.

4.5 Distance intuition

Logical operators descend from codewords and transpose-code codewords of the classical inputs. Under the usual nontriviality conditions, the HGP distance is controlled by the smallest relevant input distance. Families built from suitable classical LDPC codes can achieve constant rate and distance proportional to $\sqrt{n}$ [3]. The exact distance of a finite instance should be computed or bounded rather than inferred only from asymptotic statements.

The construction in one sentence

The HGP takes two sparse classical constraint systems, creates qubits from variable-variable and check-check pairs, and uses Kronecker products to produce sparse commuting quantum checks.

5 Worked example: the project fixture [[13, 1, 3]]

5.1 Classical input

Use the length-three repetition check for both inputs:

$$H_1 = H_2 = H = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}. \quad (25)$$

Here $n_1 = n_2 = 3$, $r_1 = r_2 = 2$, $\text{rank}(H) = 2$, $k_1 = k_2 = 1$, and $k_1^T = k_2^T = 0$.

5.2 Derived parameters

The HGP formulas immediately give

$$n = 3 \cdot 3 + 2 \cdot 2 = 13, \quad (26)$$

$$k = 1 \cdot 1 + 0 \cdot 0 = 1. \quad (27)$$

The construction produces six X -check rows and six Z -check rows. For this fixture both check matrices have rank six, so $k = 13 - 6 - 6 = 1$. Exact enumeration gives $d_X = d_Z = 3$, hence

$$[[n, k, d]] = [[13, 1, 3]]. \quad (28)$$

Property	Meaning	Value
Physical qubits n	Columns shared by H_X, H_Z	13
Logical qubits k	Protected degrees of freedom	1
Distance d	Minimum nontrivial logical Pauli weight	3
X checks	Rows of H_X	6
Z checks	Rows of H_Z	6
Check weights	Number of qubits per generator	3 or 4
Guaranteed correction	All errors through weight $\lfloor (d-1)/2 \rfloor$	1

5.3 Why this is a useful teaching code

- It is small enough to show every qubit, check, and nonzero syndrome.
- Its construction still exercises the real HGP algebra and two qubit sectors.
- Exact distance and exhaustive decoding are feasible, providing an oracle for tests.
- Weight-three logical operators make success and failure examples easy to construct.
- The code is not presented as a performance benchmark for large-scale fault tolerance.

Project connection

The Configure stage already exposes this fixture and its calculated validation evidence. Inject, Observe, and Decode should consume the same immutable code identifier and matrix ordering. A user selection such as “qubit 7 receives Y ” is meaningful only if all stages share that ordering.

6 Noise models and error injection

6.1 Code-capacity noise

The first project model assumes perfect stabilizer measurements and applies noise only to data qubits. This is the code-capacity setting. For independent depolarizing noise,

$$\Pr(I) = 1 - p, \quad \Pr(X) = \Pr(Y) = \Pr(Z) = \frac{p}{3}. \quad (29)$$

Alternative independent channels can assign different probabilities p_X, p_Y, p_Z as long as they sum to at most one.

The model is intentionally simple. It isolates code and decoder behavior from measurement-circuit design, repeated rounds, leakage, and time correlations.

6.2 Manual injection

Manual mode should let a learner choose qubits and Pauli types directly. The internal state is the pair (e_X, e_Z) . Adding another Pauli on the same qubit uses Pauli multiplication up to phase, which becomes XOR in the binary representation. For example,

$$X \cdot Z \sim Y, \quad (1, 0) + (0, 1) = (1, 1). \quad (30)$$

The interface may either replace the selected Pauli or explicitly compose it, but the rule must be visible and deterministic.

6.3 Seeded stochastic injection

A reproducible run stores at least

$$(\text{code version, noise model, } p, \text{seed, generator version}). \quad (31)$$

The seed alone is insufficient if sampling order or pseudorandom generator behavior later changes. The generated error should be stored as part of the run record so that a historical result remains inspectable.

6.4 Later noise-model layers

Model	Adds	Why it is harder
Code capacity	Data-qubit Pauli errors; perfect measurements	One syndrome snapshot; suitable for the first implementation
Phenomenological	Data errors plus noisy syndrome bits across rounds	Decoder must use space-time information
Circuit level	Faults on gates, preparation, idling, and measurement	A single fault can propagate to correlated data and measurement errors
Biased or correlated	Unequal Pauli rates or multi-qubit events	Independent binary decoding may discard useful correlations
Erasures	Error locations are known, values may be unknown	Different likelihood model and often easier combinatorics

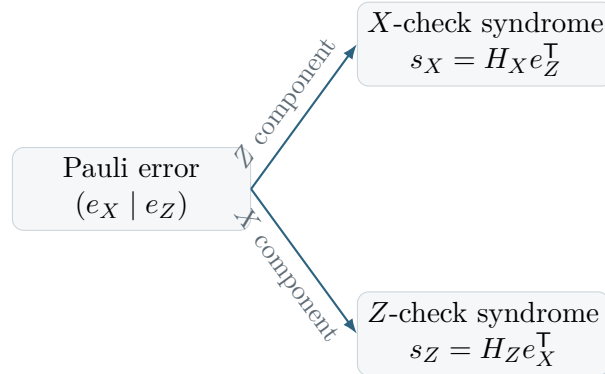
Common pitfall

A threshold is never just a property of a check matrix. It belongs to a combination of code family, decoder, noise model, syndrome-extraction model, and scaling experiment.

7 Syndromes: what the system observes

7.1 Calculating the two syndrome channels

Given (e_X, e_Z) , calculate the syndromes using Equation (9). Each syndrome bit is the parity of error components adjacent to that check. In graph language, a check is activated when it has odd overlap with the anticommuting component of the error.



7.2 What a syndrome does and does not reveal

The syndrome reveals an equivalence class of candidate errors. If e and e' differ by an element of the kernel of the relevant check matrix, then they have the same syndrome. In a quantum code, same-syndrome candidates can differ by:

- a stabilizer, in which case they act identically on logical information; or
- a logical operator, in which case choosing the wrong class causes logical failure.

This is why syndrome matching is necessary but not sufficient for successful decoding.

7.3 Observe-stage representations

The project should present the same syndrome through linked views:

- a compact binary vector for exactness and export;
- a check list with satisfied and violated constraints;
- a Tanner-graph view that highlights local neighborhoods; and
- a matrix view that shows the parity calculation for a selected check.

Selecting a check in one view should highlight the same check and neighboring qubits in the others. This supports recognition, causal reasoning, and debugging without adding scientific state.

8 The quantum decoding problem

8.1 Decoder input and output

For one CSS component, a decoder receives a check matrix H , syndrome s , and a noise prior. It returns a candidate correction $\hat{e}$ satisfying

$$H\hat{e}^T = s. \quad (32)$$

For the full CSS code, corrections $(\hat{e}_X, \hat{e}_Z)$ are produced from the two syndrome channels with consistent conventions.

8.2 The real success condition

Let E be the actual error and R the proposed recovery. Decoding succeeds when

$$RE \in \mathcal{S}, \quad (33)$$

up to an irrelevant phase. In binary addition, the residual error $e + \hat{e}$ must be a stabilizer. It need not be the zero vector. This distinction is degeneracy.

Outcome	Syndrome test	Logical test
Invalid correction	$H\hat{e}^T \neq s$	Decoder failed to return a feasible correction
Successful recovery	$H\hat{e}^T = s$	Residual is a stabilizer
Logical failure	$H\hat{e}^T = s$	Residual is a nontrivial logical operator

8.3 Maximum-likelihood objectives

A nondegenerate decoder might choose the most likely physical error:

$$\hat{e}_{\text{physical}} = \arg \max_{e: He^T = s} \Pr(e | s). \quad (34)$$

A degenerate maximum-likelihood decoder instead chooses the most likely logical equivalence class:

$$\hat{\mathcal{C}} = \arg \max_{\mathcal{C}} \sum_{e \in \mathcal{C}} \Pr(e | s). \quad (35)$$

The second objective is the quantum-correct one, but it is generally more difficult. Practical decoders approximate it in different ways.

8.4 Two independent binary decoders or one correlated decoder?

For independent X and Z noise, a CSS decoder can treat the components separately. Under depolarizing noise, a Y error couples both components, so separate decoding discards correlation. The first project implementation can use separate channels because it makes every step legible. A later correlated decoder should be evaluated against the same run records.

9 Decoder families relevant to HGP qLDPC codes

9.1 Exact lookup or exhaustive decoding

For the $[[13, 1, 3]]$ fixture, enumerate errors or syndrome-consistent candidates and classify their logical cosets. Exact decoding is exponentially expensive in n , but it provides:

- an interpretable first decoder;
- a correctness oracle for optimized decoders;
- exact examples of degeneracy and tie breaking; and
- deterministic traces suitable for teaching.

A useful baseline selects a minimum-weight candidate, applies a deterministic lexicographic tie break, verifies Equation (32), and reports the residual logical class.

9.2 Belief propagation

Belief propagation (BP) passes probabilistic messages along the Tanner graph. For a binary symmetric channel with crossover probability p , the prior log-likelihood ratio is

$$L_j^{(0)} = \log \frac{1-p}{p}. \quad (36)$$

In sum-product BP, a check-to-variable update has the form

$$L_{a \rightarrow j} = 2 \operatorname{atanh} \left((-1)^{s_a} \prod_{j' \in N(a) \setminus j} \tanh \frac{L_{j' \rightarrow a}}{2} \right), \quad (37)$$

and a variable-to-check update combines the prior with other incoming checks:

$$L_{j \rightarrow a} = L_j^{(0)} + \sum_{b \in N(j) \setminus a} L_{b \rightarrow j}. \quad (38)$$

On a tree, BP gives exact marginals. qLDPC Tanner graphs contain cycles, and quantum degeneracy can create symmetric or split beliefs. BP may oscillate, converge to an inconsistent hard decision, or settle on a poor logical class.

9.3 BP with ordered-statistics decoding

BP+OSD uses BP to estimate bit reliabilities, then invokes linear-algebraic post-processing when a reliable syndrome-consistent answer is needed. It has demonstrated broad utility for HGP qLDPC codes [5].

At a high level:

1. run BP for a fixed number of iterations or until a stopping rule;
2. sort columns by reliability;

3. select an information or pivot structure through elimination;
4. construct a syndrome-consistent baseline candidate; and
5. test low-order flips among selected unreliable positions, retaining the best metric.

OSD order controls a performance-complexity tradeoff. Order zero tests the elimination-derived candidate. Higher orders enumerate more perturbations. The decoder trace should record BP iterations, final reliabilities, permutation, OSD order, candidates tested, and selection metric.

9.4 Small-set-flip

Small-set-flip is designed for quantum expander codes, a structured subclass of HGP codes. It repeatedly chooses a small subset supported within a stabilizer neighborhood whose flip reduces the syndrome sufficiently. Under expansion assumptions it has strong provable correction and runtime guarantees [4].

The $[[13, 1, 3]]$ teaching fixture should not be described as satisfying those asymptotic expansion guarantees without checking the hypotheses. The algorithm remains valuable conceptually because its progress measure is visible: each accepted local flip changes the residual syndrome.

9.5 Other useful directions

- **BP with guided decimation:** progressively fixes highly reliable variables to break symmetry and encourage convergence [8].
- **Localized statistics decoding:** performs inversion-based post-processing on local clusters rather than one global system.
- **Generalized union-find:** grows and merges syndrome regions; extensions have been studied for broader qLDPC families [6].
- **Integer or tensor-network methods:** can provide strong baselines on limited sizes, usually at greater computational cost.

Recommended implementation sequence

Start with exact minimum-weight lookup for the small fixture, because it makes correctness observable. Add BP with a complete message trace. Then add OSD post-processing and compare it to the exact oracle. This sequence separates implementation errors from algorithmic limitations.

10 A detailed BP+OSD decoding trace

This section defines the information the Decode stage should expose for one binary component.

10.1 Inputs

- check matrix H and its stable row and column ordering;
- observed syndrome s ;

- channel prior p or per-qubit priors;
- BP update rule, schedule, damping factor, and maximum iterations;
- OSD method, order, and candidate metric; and
- deterministic tie-breaking policy.

10.2 BP phase

For each iteration, retain summary values rather than every transient UI animation:

1. compute check-to-variable messages from the previous variable messages;
2. compute variable-to-check messages and posterior beliefs;
3. form a hard decision $\hat{e}^{(t)}$;
4. compute the predicted syndrome $H\hat{e}^{(t)\top}$;
5. record whether the target syndrome is satisfied; and
6. stop according to the declared rule.

Useful diagnostics include maximum message change, number of unsatisfied checks, hard-decision weight, and whether a state repeated. Message clipping and damping must be reported because they can materially change convergence.

10.3 OSD phase

OSD consumes BP reliabilities, but its output must still be verified independently. The trace should show:

- reliability-ranked qubits;
- the column permutation used for elimination;
- pivot and free positions;
- the baseline candidate and each tested perturbation class;
- the candidate cost under the noise model; and
- the final syndrome-consistency result.

10.4 Outcome classification

After combining the X and Z corrections, compute the residual $E + R$. Report three separate facts:

1. **feasibility:** did the correction reproduce the observed syndrome?
2. **logical result:** is the residual a stabilizer or a nontrivial logical operator?
3. **effort:** how many iterations, candidates, and units of time were used?

The user should never have to infer logical success from a green “converged” label. Convergence and correctness are different.

11 Project architecture and data contracts

11.1 Scientific state model

A run should be a versioned record rather than a loose collection of interface fields. A compact conceptual schema is:

```
run = {
  code: {id, version, Hx, Hz, parameters, ordering},
  model: {noise family, probability, measurement assumptions},
  injection: {mode, seed, Pauli error},
  observation: {sx, sz, convention},
  decoding: {decoder, settings, correction, trace},
  outcome: {syndrome consistent, logical class, metrics}
}
```

Derived values should be recomputable from earlier state. For example, the Observe stage should not accept an arbitrary syndrome as if it came from the stored error unless the run is explicitly in a manual-syndrome mode.

11.2 Stage contracts

Stage	Inputs	Outputs	Core invariant
Configure	Code identifier, noise choice, parameters, seed	Confirmed immutable configuration	Code evidence and assumptions are valid
Inject	Confirmed configuration	Explicit (e_X, e_Z) and Pauli labels	Binary and Pauli views agree
Observe	Code matrices and error	(s_X, s_Z) plus check states	Syndromes equal matrix products
Decode	Matrices, syndrome, priors, decoder settings	Correction, trace, residual class	Output is independently verified

11.3 API responsibilities

The backend should own scientific calculations and validation. The frontend should own interaction, linked views, and presentation. This division avoids two independent implementations silently disagreeing. Suggested service boundaries are:

- GET `/api/codes`: list verified code fixtures and summaries;
- GET `/api/codes/{id}`: retrieve matrices, parameters, conventions, and evidence;

- `POST /api/injections`: validate or generate a reproducible Pauli error;
- `POST /api/syndromes`: calculate syndromes and per-check explanations;
- `POST /api/decodings`: run a named decoder with explicit settings; and
- `GET /api/runs/{id}`: retrieve a complete reproducible record when persistence is added.

These are design targets, not claims that every endpoint is already implemented.

12 Human-centered scientific interaction

The HCI objective is not to simplify away the mathematics. It is to control complexity while preserving causal and scientific meaning.

12.1 Interaction principles

- **Progressive disclosure**: show the decision first, then matrices, traces, and proofs on demand.
- **Visibility of state**: distinguish draft, confirmed, generated, observed, decoding, and verified states.
- **Recognition over recall**: label Pauli types, check conventions, and parameter meanings near the control that uses them.
- **Error prevention**: constrain probabilities, seeds, qubit indices, and incompatible settings before execution.
- **Reversibility**: allow edits by returning to the earliest affected stage and clearly invalidating downstream results.
- **Multiple coordinated representations**: link matrix, graph, vector, and textual explanations to one canonical state.
- **Honest uncertainty**: attach confidence intervals and sample counts to empirical performance claims.

12.2 Visual semantics

Color should be redundant, not exclusive. An activated check needs a symbol and text label in addition to color. X , Y , and Z should have stable encodings across all stages. Focus order, keyboard operation, sufficient contrast, and screen-reader labels are part of scientific usability: a user who cannot access a graph must still access the same check and syndrome data.

12.3 Details on demand

A well-layered Decode result might show:

1. first: logical success or failure, correction weight, and runtime;
2. next: syndrome consistency and residual class;

3. then: iteration-level decoder diagnostics; and
4. finally: raw messages, permutations, candidate lists, and exported run data.

This ordering serves both a learner asking “what happened?” and a researcher asking “why?” without creating separate products.

13 Experimental methodology

13.1 Single-run diagnostics

One run is useful for explanation and debugging. Record:

- actual error weight and Pauli composition;
- syndrome weights $\text{wt}(s_X)$ and $\text{wt}(s_Z)$;
- correction weight;
- residual stabilizer or logical class;
- decoder convergence, iterations, and candidate count; and
- wall-clock time measured under a declared environment.

13.2 Monte Carlo experiments

At each physical error rate p , run N independent trials and count F logical failures. The estimated logical error rate is

$$\hat{p}_L = \frac{F}{N}. \quad (39)$$

Always report N and an interval estimate. When F is small, the naive normal interval is poor; a Wilson or exact binomial interval is more appropriate.

To compare decoders fairly, reuse the same sampled errors when possible. Paired trials reduce noise in the comparison and make disagreements directly inspectable.

13.3 Threshold studies

A threshold study requires a family of increasing code sizes and curves of logical error rate versus physical error rate under fixed decoder and noise definitions. The small $[[13, 1, 3]]$ fixture alone cannot establish a threshold. It can validate the experiment pipeline before larger HGP instances are added.

13.4 Metrics beyond logical error rate

- throughput and latency distributions, not only averages;
- memory usage and message count;
- fraction of outputs that are syndrome inconsistent;

- convergence rate and iteration distribution;
- performance by error weight or syndrome weight;
- failure-mode taxonomy: nonconvergence, wrong logical class, timeout, or invalid result; and
- sensitivity to decoder hyperparameters and seed.

14 Verification and testing strategy

14.1 Code-construction invariants

Every registered code should satisfy automated checks for:

1. binary entries and compatible dimensions;
2. CSS orthogonality $H_X H_Z^T = 0$;
3. claimed ranks and $k = n - r_X - r_Z$;
4. bounded check and qubit degrees for the intended family;
5. exact or independently justified distance for small fixtures; and
6. stable row and column ordering with a version identifier.

14.2 Stage-level properties

Component		Property test
Pauli	conver-	Pauli $\leftrightarrow (e_X, e_Z)$ round trip is exact
sion		
Injection		Same code, model, probability, generator version, and seed reproduce the same error
Syndrome		Per-check parity view equals the matrix product
Correction		Every accepted correction reproduces the input syndrome
Logical classifier		Stabilizers classify as success; known logicals classify as failure
Exact decoder		Corrects every error of weight at most one for the $[[13, 1, 3]]$ fixture
BP+OSD		Matches the exact oracle on a regression corpus where expected behavior is fixed
Serialization		Exported and reloaded runs preserve all scientific fields

14.3 Independent verification

Whenever practical, calculate important claims by two paths. For example, obtain the fixture distance by exhaustive logical search and separately verify that all weight-one errors are correctable while a weight-three logical exists. A decoder should not be responsible for grading its own output; residual classification belongs to a separate verifier.

Definition of done for a scientific slice

A stage is complete when its state contract, calculations, UI, accessibility, deterministic examples, automated tests, and failure messages agree. A visually complete screen without a verified scientific transition is not complete.

15 Implementation roadmap

15.1 Current foundation: Configure

- versioned qLDPC fixture registry;
- verified $[[13, 1, 3]]$ HGP construction and evidence;
- manual or seeded code-capacity setup; and
- confirmed configuration state shared with later stages.

15.2 Next slice: Inject

- canonical Pauli-error representation and validation;
- interactive 13-qubit selection with I, X, Y, Z states;
- seeded independent-noise generation;
- error summary, weight, and reproducibility metadata; and
- invalidation of downstream results when the error changes.

15.3 Then: Observe

- backend syndrome calculation from stored matrices and error vectors;
- separated X -check and Z -check channels with explicit naming;
- linked check-list, matrix, and Tanner-graph views;
- selected-check parity explanation; and
- syndrome invariants and export.

15.4 Then: Decode

- exact minimum-weight decoder for the teaching fixture;
- independent correction verifier and logical-outcome classifier;
- stepwise trace and counterexamples illustrating degeneracy;
- BP implementation with configurable schedule, damping, and stopping; and
- OSD post-processing with reliability and candidate inspection.

15.5 Research extensions

After the single-run workflow is correct, add larger HGP families, batch experiments, confidence intervals, decoder comparisons, and performance profiling. Phenomenological or circuit-level noise should be a later architectural extension because it changes the data model from one syndrome snapshot to a history of noisy measurements.

16 Study questions and practical exercises

16.1 Concept checks

1. Why does $H_X H_Z^T = 0$ encode commutation?
2. Why can two different errors with the same syndrome lead to different logical outcomes?
3. What information is missing from the notation $[[n, k, d]]$?
4. Why does a Y error contribute to both binary error components?
5. Why is decoder convergence not equivalent to decoding success?
6. Why can the $[[13, 1, 3]]$ code validate a pipeline but not establish a threshold?

16.2 Matrix exercises

1. Verify that the length-three repetition check has rank two and kernel $\{000, 111\}$.
2. Use Equations (16)–(17) to derive the dimensions of the two project check matrices.
3. Pick one single-qubit X error and calculate its Z -check syndrome from the matrix in Appendix B.
4. Pick one single-qubit Y error and calculate both syndrome channels.
5. Find two distinct syndrome-consistent candidates and determine whether their difference is a stabilizer or logical operator.

16.3 Implementation exercises

1. Write a property test that every manual Pauli selection round-trips through the binary representation.
2. Create a deterministic lookup table from syndrome to minimum-weight correction for one CSS component.
3. Construct a failing example in which a syndrome-consistent correction leaves a logical residual.
4. Compare BP hard decisions before and after damping on one cyclic Tanner graph.
5. Design an export record that is sufficient to replay a historical run after the codebase changes.

A Notation and glossary

Term or symbol	Meaning
$\mathbb{F}_2$	Binary field; addition and multiplication are modulo two
H	Classical parity-check matrix
H_X, H_Z	Binary support matrices for CSS X - and Z -type stabilizer generators
$[[n, k, d]]$	Quantum block length, number of logical qubits, and code distance
Syndrome	Outcomes identifying which stabilizer checks anticommute with an error
Stabilizer	Pauli operator that acts trivially on every encoded state
Logical operator	Pauli in the stabilizer normalizer but not in the stabilizer group
Degeneracy	Different physical errors or corrections that have the same logical action
Tanner graph	Bipartite graph connecting check nodes to variable or qubit nodes
qLDPC	Quantum code family with sparse, bounded-weight parity checks
HGP	Hypergraph-product construction from two classical parity-check matrices
Code-capacity noise	Data-qubit noise with perfect stabilizer measurement
BP	Belief propagation message passing on a factor or Tanner graph
OSD	Ordered-statistics post-processing using reliabilities and elimination
Residual	Product, or binary sum, of actual error and proposed recovery
Logical failure rate	Probability that the residual contains a nontrivial logical action

B Explicit matrices for the $[[13, 1, 3]]$ fixture

The following matrices use the project construction's current Kronecker ordering. Columns 1–9 belong to the $V_1 \times V_2$ sector and columns 10–13 belong to the $C_1 \times C_2$ sector.

$$H_X = \left[\begin{array}{ccccccccc|cccc} 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \end{array} \right]. \quad (40)$$

$$H_Z = \left[\begin{array}{ccccccccc|cccc} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \end{array} \right]. \quad (41)$$

The row weights of H_X are $(3, 4, 3, 3, 4, 3)$ and those of H_Z are $(3, 3, 4, 4, 3, 3)$. Direct multiplication verifies $H_X H_Z^\top = 0$.

C Decoder comparison worksheet

Decoder	Main strength	Main limitation	Evidence to expose
Exact lookup	Ground-truth behavior on tiny codes	Exponential scaling	Candidates, tie break, residual class
BP	Sparse and fast per iteration	Cycles and degeneracy can prevent useful convergence	Messages, beliefs, unsatisfied checks
BP+OSD	Robust syndrome-consistent post-processing	Elimination and candidate cost grow with order	Reliability ordering, pivots, candidates
Small-set-flip	Local steps and strong expander-code theory	Guarantees require expansion hypotheses	Selected flips and syndrome reduction
Guided decimation	Breaks BP symmetry with simple decisions	Sequential fixing can lock in mistakes	Fix order and confidence at each step
Union-find variants	Cluster-based and potentially efficient	General qLDPC behavior depends on structure	Cluster growth, merges, final correction

D Pre-experiment checklist

- ☐ Code identifier and version are recorded.
- ☐ H_X, H_Z , row ordering, and qubit ordering are fixed.
- ☐ Orthogonality, ranks, k , and available distance evidence pass.
- ☐ Noise and measurement models are stated.
- ☐ Probability convention and random generator version are stated.
- ☐ Seed and realized physical error are stored.
- ☐ Syndrome naming convention is explicit.
- ☐ Decoder version and every behavior-changing setting are stored.
- ☐ Decoder output is checked for syndrome consistency.
- ☐ Residual is independently classified as stabilizer or logical.
- ☐ Trial count and uncertainty accompany aggregate rates.
- ☐ Unsupported threshold or hardware claims are not inferred.

References

- [1] A. R. Calderbank and P. W. Shor, “Good quantum error-correcting codes exist,” *Physical Review A*, 54, 1098–1105, 1996. <https://doi.org/10.1103/PhysRevA.54.1098>
- [2] A. M. Steane, “Error correcting quantum code,” *Physical Review Letters*, 77, 793–797, 1996. <https://doi.org/10.1103/PhysRevLett.77.793>
- [3] J.-P. Tillich and G. Zemor, “Quantum LDPC codes with positive rate and minimum distance proportional to the square root of the blocklength,” *IEEE Transactions on Information Theory*, 60(2), 1193–1202, 2014. <https://doi.org/10.1109/TIT.2013.2292061>
- [4] A. Leverrier, J.-P. Tillich, and G. Zemor, “Quantum expander codes,” *56th IEEE Symposium on Foundations of Computer Science*, 810–824, 2015. <https://arxiv.org/abs/1504.00822>
- [5] J. Roffe, D. R. White, S. Burton, and E. T. Campbell, “Decoding across the quantum low-density parity-check code landscape,” *Physical Review Research*, 2, 043423, 2020. <https://doi.org/10.1103/PhysRevResearch.2.043423>
- [6] N. Delfosse, V. Londe, and M. E. Beverland, “Toward a union-find decoder for quantum LDPC codes,” 2021. <https://arxiv.org/abs/2103.08049>
- [7] P. Panteleev and G. Kalachev, “Asymptotically good quantum and locally testable classical LDPC codes,” 2021. <https://arxiv.org/abs/2111.03654>
- [8] H. Yao, W. Abu Laban, C. Hager, A. Graell i Amat, and H. D. Pfister, “Belief propagation decoding of quantum LDPC codes with guided decimation,” 2023. <https://arxiv.org/abs/2312.10950>